CODESPECT

Fraxswap Router Multihop V2

SECURITY ASSESSMENT REPORT

26 October 2026

Prepared for:

Frax Finance

Contents

1 About CODESPECT 2

2 Disclaimer 2

3 Risk Classification 3

4 Executive Summary 4

5 Audit Summary 5
- 5.1 Engagement Overview . . . 5
- 5.2 Scope – Audited Files . . . 5
- 5.3 Findings Overview . . . 6

6 System Overview 7

7 Findings 10
- 7.1 Medium . . . 10
  - 7.1.1 [Medium] Fee on transfer and stETH style tokens cannot be used as step input, despite the natspec . . . 10
- 7.2 Informational . . . 11
  - 7.2.1 [Info] Native routes break on chains without a wrapped native token, because WETH9 calls are never skipped . . . 11
  - 7.2.2 [Info] `swapWithPermit(...)` only supports EIP2612 permits with the swap deadline . . . 12
  - 7.2.3 [Info] UniV4 step calls take even when the output amount is zero . . . 12
  - 7.2.4 [Info] Misleading parameter name for executeVirtualOrders . . . 12

8 Threat Model 13

9 Evaluation of Provided Documentation 16
- 9.1 Documentation QA . . . 16

10 Test Suite Evaluation 18
- 10.1 Compilation Output . . . 18
- 10.2 Tests Output . . . 18
- 10.3 Coverage . . . 18
- 10.4 Overall Strengths . . . 18
- 10.5 Cross-Cutting Weaknesses . . . 18
- 10.6 Specific Untested Paths . . . 19
- 10.7 Conclusion . . . 19

1 About CODESPECT

CODESPECT is a specialised smart contract security firm dedicated to ensuring the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to building secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialise in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralised applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialised cybersecurity solutions designed to minimise risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty program to incentivise external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivised.
- **Medium** - Likely only under certain conditions or moderately incentivised.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilises one other category for findings: **Informational**.

a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally, including cases where certain portions of the code deviate from established smart contract development standards or best practices.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for Frax Finance on the **Fraxswap Router Multihop V2**, a multi-hop swap router that executes caller-authored routes across Uniswap V2-style pairs, Uniswap V3-style pools, Uniswap V4 and generic contract calls.

Audit Conclusion

The in-scope contracts were assessed by the CODESPECT team through manual review, static analysis, and dynamic testing of route execution. No `Critical` or `High` severity issue was identified, and the most impactful findings were resolved by Frax during the fix review. The findings that remain acknowledged describe incompatibility with particular chains and particular situations rather than a flaw in the routing logic: a chain without a wrapped native token, a token outside EIP-2612, and a route whose output rounds down to zero. Each of them can be mitigated by following the updated documentation and by confirming the deployment configuration of every target chain before launch. Because routes are built outside the audited code, integrators who build their own should validate them against the checks the documentation describes. The Frax team was very responsive throughout the review and answered all of our questions. With the fix review complete and no finding left unresolved, we consider the in-scope contracts ready for production deployment, subject to the operational checks noted above.

The audit was performed using:

a) Manual analysis of the codebase.

b) Static analysis of the in-scope contracts.

c) Documentation review, cross-checking the natspec and the shipped documentation against the code.

d) Dynamic analysis of the smart contracts and execution testing, including fuzzing of route construction and fork tests against Uniswap V4.

CODESPECT identified **5 points of attention**: **0** classified as `Critical`, **0** as `High`, **1** as `Medium`, **0** as `Low`, and **4** as `Informational`. All issues are summarised in the table below and detailed in Section 7.

Organisation of the document is as follows:

- **Section 3** presents the risk classification methodology and reference frameworks.
- **Section 5** summarises the audit: scope, schedule, coverage, and auditors.
- **Section 6** describes the functionality of the code in scope.
- **Section 7** presents the findings.
- **Section 8** presents the threat model.
- **Section 9** discusses the documentation provided by the client.
- **Section 10** discusses the test suite provided by the client.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
Medium	0	1	0
Informational	0	1	3
Total	**0**	**2**	**3**

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

5.1 Engagement Overview

Audit Type	Security Review
Project Name	Fraxswap Router Multihop V2
Type of Project	Multi-hop DEX Aggregation Router
Duration of Engagement	3 days
Duration of Fix Review Phase	1 day
Initial Report	3 October 2026
Final Report	26 October 2026
Repository	FraxFinance/dev-fraxswap-v3
Commit (Audit)	`cca72784197038d57d31dd0a6b5f23c32c265c95`
Commit (Final)	`488caac2271217cf6a5b817fb7e1fc4d61f8956d`
Deployment	Ethereum mainnet
Router Address	`0x00000000580f8b07840ecbfa1c7faa8d3e6434d7`
Executor Address	`0x3548Eb1185C482C0F3eCa3c280E767c2efAe8039`
Documentation Assessment	Medium
Test Suite Assessment	Medium
Auditors	Kalogerone, talfao

Table 3: Summary of the Audit

The addresses above are the live deployment of the audited contracts on Ethereum mainnet. We confirmed on chain that the deployed router carries the fix-review code and not the code of the audited commit: the recipient guard that commit `488caac` introduced is present in the deployed bytecode. The executor address is the one the deployed router reports.

5.2 Scope – Audited Files

	Contract	LoC
1	FraxswapRouterMultihopV2.sol	500
2	PancakeV3Plugin.sol	87
	Total	**587**

5.3 Findings Overview

	Finding	Severity	Status
1	[Medium] Fee on transfer and stETH style tokens cannot be used as step input, despite the natspec	Medium	Fixed
2	[Info] Native routes break on chains without a wrapped native token, because WETH9 calls are never skipped	Info	Acknowledged
3	[Info] `swapWithPermit(...)` only supports EIP2612 permits with the swap deadline	Info	Acknowledged
4	[Info] UniV4 step calls take even when the output amount is zero	Info	Acknowledged
5	[Info] Misleading parameter name for executeVirtualOrders	Info	Fixed

6 System Overview

Fraxswap Router Multihop V2 is a swap aggregation router that executes a caller-supplied *route* across several on-chain venues in a single transaction. The scope comprises four contracts in two files: `FraxswapRouterMultihopV2`, `FraxswapExecutorV2` and `FraxswapRouteEncoder` (all in `FraxswapRouterMultihopV2.sol`), and `PancakeV3Plugin` (in `PancakeV3Plugin.sol`).

Architecture: Router and Executor

The system is split into two contracts with different privileges. `FraxswapRouterMultihopV2` is the user-facing contract and the only one that holds user token approvals. `FraxswapExecutorV2` runs the route and never holds user approvals. Its route entry point, `execute(...)`, is restricted by `onlyRouter`.

The split exists so that route steps, including the generic call step, can call arbitrary contracts from an address that has nothing to steal. The executor temporarily holds the in-flight funds. It measures the final output itself, checks it against the user's minimum, and pays the recipient. The executor also accepts native ETH and ERC-721 and ERC-1155 tokens, for purposes such as the KYC gating of compliance tokens like WTGXX.

Router entry points and funding

The router exposes two payable entry points:

```
function swap(FraxswapParams calldata params)
    external
    payable
    nonReentrant
    checkDeadline(params.deadline)
    returns (uint256 amountOut)

function swapWithPermit(FraxswapParams calldata params, bool approveMax, uint8 v, bytes32 r, bytes32 s)
    external
    payable
    nonReentrant
    checkDeadline(params.deadline)
    returns (uint256 amountOut)
```

`swapWithPermit(...)` first calls the input token's EIP-2612 `permit(...)`, tolerating a failure, and then proceeds as `swap(...)`. Both call the internal `_swap(...)`, which takes the input from the caller, hands it to the executor and returns the output that the executor measured. When the route sets direct funding, the input goes straight to the first pool instead.

Route format

The swap parameters are passed in a `FraxswapParams` struct, whose `route` field is a packed byte string that the router forwards to the executor without decoding it.

```
struct FraxswapParams {
    address tokenIn;          // address(0) for native ETH
    uint256 amountIn;
    address tokenOut;         // address(0) for native ETH
    uint256 amountOutMinimum; // must be > 0
    address recipient;
    uint256 deadline;
    bytes route;              // packed route, see below
}
```

```
route := [header u8] swap*
         header bits 0-6: number of swaps, bit 7: fund the first step's target directly

swap  := [tokenIn 20B] [flags u8] step*
         flags bits 0-6: number of steps, bit 7: use the executor's measured balance as basis

step  := [pctBps u16] [stepType u8] [creditTo u8] [target 20B] [dataLen u16] [data]
         creditTo: index of the swap credited with the step's output, 0xFF for none
```

A route is an ordered list of swaps over a token graph. Each step takes `pctBps` of its swap's input and credits its output to the swap named by `creditTo`. These credits only size later inputs: the final output is always measured by the executor.

Venue steps

`executeStep(...)` dispatches on `stepType` to one of four venue types:

- **UniV2-style pool** (0) – sends the input to a pair and calls its `swap(...)`. The output is computed from the reserves or from the pair's own `getAmountOut(...)`, and flags select the direction, fee, receiver and whether the Fraxswap TWAMM orders are executed first.
- **UniV3-style pool** (1) – calls the pool's `swap(...)` with an exact input and pays it from `uniswapV3SwapCallback(...)`, which accepts only the pool armed for the current step.
- **Generic call** (2) – calls an arbitrary target with encoded call data, optionally patching the step input into it and funding the target by approval, transfer or native value. It is the extension point for venues without a dedicated step.
- **UniV4** (3) – unlocks the Uniswap V4 `PoolManager`, then swaps, settles and takes inside `unlockCallback(...)`.

Final output check and payout

The executor's route entry point is restricted to the router:

```
function execute(
    bytes calldata route,
    uint256 amountIn,
    address tokenOut,
    address payable recipient,
    uint256 amountOutMinimum
) external payable onlyRouter returns (uint256 amountOut) {...}
```

`execute(...)` runs the route and then measures the executor's balance of `tokenOut`, unwrapping WETH for a native output. It requires the amount to be at least `amountOutMinimum` and pays all of it to the recipient. The slippage protection is therefore the measured balance, not the amounts reported by the steps.

Route encoder

`FraxswapRouteEncoder` is a stateless helper, not called by the router, whose pure functions assemble the packed route format for tooling and tests. It enforces only a few length limits and does not validate routes.

PancakeV3Plugin

PancakeSwap V3 pools pay through `pancakeV3SwapCallback(...)`, which the executor does not implement. `PancakeV3Plugin` supplies it and is driven as a generic call step, so the router and executor need no change. The plugin has no owner.

```
function swap(
    address pool,
    address tokenIn,
    uint256 amountIn,
    bool zeroForOne,
    address recipient
) external returns (uint256 amountOut) {...}

function getAmountOut(
    address pool,
    address tokenIn,
    uint256 amountIn,
    bool zeroForOne
) external returns (uint256 amountOut) {...}

function pancakeV3SwapCallback(int256 amount0Delta, int256 amount1Delta, bytes calldata data) external {...}
```

`swap(...)` swaps against the pool and returns the output, refunding any unused input to the caller. `getAmountOut(...)` is a quoting function meant for `eth_call`. The callback pays the pool only for the swap in flight.

Trust model and roles

The contracts define no privileged roles and are not upgradeable. The swapper supplies the route, amounts, minimum output, recipient and deadline. Venues and route targets are untrusted and are reached only from the executor, which holds no user approvals.

According to the client, routes are built by its API and users review the non-route parameters before signing. On-chain, the route is not validated, and the `amountOutMinimum` check on the measured balance is the protection.

System Flow

a. The client's API builds the route, and the swapper approves the router or signs a permit.

b. The swapper calls `swap(...)` or `swapWithPermit(...)`. The router checks the deadline and moves the input to the executor, or to the first pool when direct funding is set.

c. The executor runs the route step by step and credits each step's output to later swaps.

d. The executor measures the output, checks it against `amountOutMinimum` and pays the recipient.

e. The router emits `Routed` and returns the amount to the caller.

7 Findings

7.1 Medium

7.1.1 [Medium] Fee on transfer and stETH style tokens cannot be used as step input, despite the natspec

File(s): `FraxswapRouterMultihopV2.sol`, `PancakeV3Plugin.sol`

Description: The natspec presents the measured balance flag of a swap as the way to support fee on transfer tokens. The flag replaces the swap's basis with `balanceOf(executor)`, which corrects how much the executor actually received. It does not change what happens when the executor forwards the tokens. Every step sends its input on to a venue, and for these tokens that transfer delivers less than the amount sent. Meanwhile, the step's amounts are still based on the full amount:

a. UniV2 steps. `stepUniV2(...)` computes `amountOut` from the step's input amount and then transfers that amount to the pair. The pair receives less, and its K check reverts. Without the flag the executor's own transfer already fails, since it sends the full amount while holding less. The same K revert happens when the router funds the first pair directly, and when one pool sends its output straight to the next pool through a receiver override. In those two cases the flag does not help either: the executor holds none of the token, so the step is skipped and the route reverts on the final output check;

b. UniV3 steps. `uniswapV3SwapCallback(...)` pays exactly the amount the pool requested. The pool receives less, and its balance check reverts with `IIA`;

c. UniV4 steps. `unlockCallback(...)` calls `sync(...)`, transfers the owed amount, and calls `settle()`. `settle()` only credits what actually arrived, so `unlock(...)` reverts with `CurrencyNotSettled`;

d. PancakeV3Plugin. The executor transfers the step's input to the plugin, which receives less.
The plugin's `pancakeV3SwapCallback(...)` then tries to pay the pool the full amount and fails;

Example with a token that charges a 1% fee on every transfer and a user swapping 100:

a. The router pulls 100 from the user, and the executor receives 99;

b. Without the flag, the swap's basis is 100. The step tries to send 100 while the executor holds 99, and reverts;

c. With the flag, the basis is 99. The step sends 99 to the pair, which receives 98.01. The step still prices the swap on 99, so the pair's K check reverts;

stETH charges no fee but is share based: `transfer(...)` converts the amount to shares and back, rounding down twice, so the recipient can receive 1 or 2 wei less than the amount sent. Lido documents this behavior. It fails the same way whenever the rounding shortchanges the venue, which happens for a large share of amounts but not for every amount. UniV3, UniV4 and PancakeSwap V3 pools revert on any shortfall. UniV2 pairs sometimes absorb one wei through the rounding of `amountOut`. With the PancakeV3Plugin, the plugin's own payment succeeds because it forwards exactly the shares it received, but the pool still receives 1 or 2 wei less and its `IIA` check reverts. There is an additional stETH case. Two steps can each credit stETH to the same later swap. The credits add up the amounts each step reported, while the executor holds the rounded down shares of each transfer. Forwarding the summed amount can need one more share than the executor holds, so the transfer reverts. The measured balance flag fixes this case.

Impact: Routes that use a fee on transfer token or a stETH style token as the input of a UniV2, UniV3 or UniV4 step, or of the PancakeV3Plugin, always revert. No funds are lost, but these tokens cannot be routed through the most common venue types. The natspec suggests otherwise. Integrators and the routing backend following it will build routes that always fail.

Recommendation(s): Consider the following:

a. For UniV2 steps, add a mode that transfers the input first, measures what the pair actually received as `balanceOf(pair)` minus the input reserve, and computes `amountOut` from that amount. This is the approach of
Uniswap's `swapExactTokensForTokensSupportingFeeOnTransferTokens(...)`;

Alternatively, allow a percentage reduction of `amountOut` that also applies when the pair's own `getAmountOut(...)` is used.

a. UniV3 and UniV4 pools require the exact amount they request, so these tokens can never be their input. Document this limitation;

b. Correct the natspec so that it describes when the measured balance flag helps and when it does not;

c. Recommend routing wstETH instead of stETH, since wstETH transfers are exact;

Status: Fixed

Update from Frax: "Non standard tokens are best effort. UniV2 reserves-formula steps like Fraxswap already support fee-on-transfer tokens through the custom fee flag (`feeBps` = pool fee + transfer fee). We can build a plugin when there is demand." Fixed for UniV2 style steps in commit `488caac`.

Update from CODESPECT: Fixed for Uni-V2. The rest is documented as unsupported.

7.2 Informational

7.2.1 [Info] Native routes break on chains without a wrapped native token, because WETH9 calls are never skipped

File(s): `FraxswapRouterMultihopV2.sol`

Description: The router takes a `WETH9` constructor argument and calls it unconditionally in two places that also handle plain native funds:

a. `execute(...)` for a native output (`tokenOut == address(0)`) always reads `IERC20(WETH9).balanceOf(executor)` and calls `WETH9.withdraw(...)` when that balance is above zero, before paying out the native balance;

b. `stepCall(...)` with fundingMode 3 accepts `tokenIn` equal to `address(0)` or `WETH9`, and calls `WETH9.withdraw(...)` when `tokenIn == address(WETH9)`;

Stable is one of the deployment chains, and it has no WETH9 style wrapper. Its gas token USDT0 is both the native asset and an ERC20 token on the same balance, and the ERC20 has no `deposit()` or `withdraw(...)`, so it cannot serve as `WETH9` either. The natural configuration on such a chain is `WETH9 = address(0)`. However, the router does not support that value:

a. In `execute(...)`, `IERC20(address(0)).balanceOf(...)` is a call to an address without code, which reverts. Every route with a native output therefore reverts, even when it never touched a wrapped token;

b. In `stepCall(...)`, a fundingMode 3 step with a native `tokenIn` satisfies `tokenIn == address(WETH9)`, since both are `address(0)`. It calls `WETH9.withdraw(...)` on `address(0)` and reverts, even though the step only needs to forward native value;

Native input on its own still works, because `_swap(...)` handles `tokenIn == address(0)` before the WETH9 branch, and ERC20 only routes are unaffected.

Impact: On Stable, and on any other deployment chain without a WETH9 style wrapper, the native asset cannot be used as the output of a route or as the value of a fundingMode 3 step, although the executor holds the native funds and needs no wrapped token to pay them out. Users of these routes get a revert.

Recommendation(s): Support `WETH9 = address(0)` for chains without a wrapped native token, and guard every wrapped native call:

a. In `execute(...)`, only read the WETH9 balance and unwrap when `address(WETH9) != address(0)`;

b. In `stepCall(...)`, only call `WETH9.withdraw(...)` when `address(WETH9) != address(0)` and `tokenIn == address(WETH9)`;

c. In `_swap(...)`, add the same check to the WETH9 with value branch for clarity;

Status: Acknowledged

Update from Frax: “We can deploy a WETH9 like contract on Stable which is then used on construction so these calls do not fail.”

Update from CODESPECT: Acknowledged. Deploying a wrapper on every target chain that lacks one keeps the unconditional calls valid, so the reverts do not occur

7.2.2 [Info] `swapWithPermit(...)` only supports EIP2612 permits with the swap deadline

File(s): `FraxswapRouterMultihopV2.sol`

Description: `swapWithPermit(...)` calls `permit(owner, spender, value, deadline, v, r, s)` inside a `try/catch`, so that a permit that was front run does not block the swap. DAI uses a different permit signature, `permit(holder, spender, nonce, expiry, allowed, v, r, s)`. For DAI the call fails, the failure is swallowed, and the swap then reverts on `transferFrom(...)` with the generic `STF` error. DAI holders therefore cannot use `swapWithPermit(...)`. There are two further limitations:

a. The permit deadline is always set to `params.deadline`, so a signature created with any other deadline fails in the same silent way;
b. Permit2 signatures are not supported;

Recommendation(s): Consider the following:

a. Add support for DAI style permits, either through a separate function or through a flag selecting the permit type;
b. Accept a separate permit deadline instead of reusing the swap deadline;
c. Consider supporting Permit2;

Status: Acknowledged

Update from Frax: "We only support the permit for tokens that support EIP2612. DAI and also many other tokens do not support EIP2612, so we do not support a permit for them. Approve is supported for all ERC20 token, so that can be used."

Update from CODESPECT: Acknowledged. EIP-2612 is the only permit standard the client intends to support, and holders of other tokens can use a plain approval followed by `swap(...)`, so no user is locked out of the router.

7.2.3 [Info] UniV4 step calls take even when the output amount is zero

File(s): `FraxswapRouterMultihopV2.sol`

Description: In `unlockCallback(...)`, the executor always calls `take` on the PoolManager for the output currency, even when `amountOut` is zero. `PoolManager.take` transfers the full amount without skipping zero. Some tokens revert on a zero value transfer, such as the legacy BNB token on Ethereum. Suppose a route sends a dust amount through a UniV4 step, for example because of a small `pctBps` split or a measured basis that only picks up dust, and the output of that step rounds down to zero. The zero value transfer then reverts, and the whole swap fails even though the rest of the route is valid. No funds are lost since the transaction reverts, but the route becomes unusable for such tokens.

Recommendation(s): Only call `take(...)` when `amountOut` is greater than zero, for example with `if (amountOut > 0)` around the call. When the output is zero, the executor has no open delta for that currency, so `unlock` still succeeds.

Status: Acknowledged

Update from Frax: "I am thinking about when this will ever be an issue. Only when the created route is so bad that the full UniV4 swap returns less than dust. In that case, the extra check only saves a little extra gas on the already fully wasted swap."

Update from CODESPECT: Acknowledged. The condition needs a step whose output rounds down to zero, which only arises on a route that would return dust in any case, so the swap carries no value for the user whether it reverts or not.

7.2.4 [Info] Misleading parameter name for executeVirtualOrders

File(s): `FraxswapRouterMultihopV2.sol`

Description: The local `PoolInterfaceV2` declares `executeVirtualOrders(uint256 blockNumber)`. `FraxswapPair` implements `executeVirtualOrders(uint256 blockTimestamp)` instead. It only executes the pending long term orders when the argument is greater than the last virtual order timestamp and not greater than `block.timestamp`.

```
// Interface used to call pool specific functions
interface PoolInterfaceV2 {
    // Fraxswap
    function executeVirtualOrders(uint256 blockNumber) external;

    function getAmountOut(uint256 amountIn, address tokenIn) external view returns (uint256);
}
```

The executor correctly passes `block.timestamp`, so the current behavior is correct.

Recommendation(s): Rename the parameter to `blockTimestamp` to match `FraxswapPair`.

Status: Fixed

Update from Frax: Fixed in commit `488caac`.

Update from CODESPECT: Fixed.

8 Threat Model

Methodology

This threat model uses **actor-capability analysis** as its spine, cross-checked against an **invariant-violation matrix**. The actor lens fits because the system has no owner, no administrator, no upgrade path and no persistent protocol state. Its risk comes from who authors the route, what the route may make the executor call, and who can reach the open surfaces that hold in-flight funds.

Each property the system should preserve is phrased as an attacker goal and matched against its mitigation. Threats carry informal STRIDE tags. The chapter is deliberately **untiered**: severity and likelihood belong to the findings.

Out of Scope

Only `FraxswapRouterMultihopV2.sol` and `PancakeV3Plugin.sol` are in scope. The venues the executor calls (Uniswap V2-style pairs, V3 and PancakeSwap V3 pools, the V4 `PoolManager` and its hooks, and generic call targets), the token contracts and the imported libraries are external black boxes. Only the in-scope contracts' interaction with them is modelled.

The client's off-chain route builder is a trusted party, and the quoter is not in scope. Both are operational assumptions, but the on-chain effect of the route actually supplied is still modelled, because the contracts do not authenticate who authored it. Chain-level facts such as transient storage support and the wrapped native token address are deployment assumptions.

Assets

a. **The swapper's input and the output owed to the recipient.** The goal of an attacker is to make the swapper receive less than the route promised, or to divert the proceeds.

b. **User token approvals on the router.** The design keeps these away from the executor, where untrusted calls happen.

c. **Funds in flight inside the executor.** One shared executor holds every swap's input and intermediate balances, so tokens of one swap could be mis-attributed to another.

d. **Residual balances and approvals on the executor and the plugin.** Leftovers are documented as finders keepers.

e. **The KYC credential held by the executor.** Compliance-gated tokens such as WTGXX can be received only because the shared executor holds a KYC NFT.

f. **Liveness of the swap path.** A hostile route, hook or recipient can deny a single swap, but no persistent state can brick later ones. The one shared executor address can, however, be blacklisted or frozen by a token issuer.

Trust Boundaries

- **Executor ↔ route-chosen targets.** This is the dominant boundary. No allow-list or code check applies to any target the route names. Safety rests on the final `amountOutMinimum` test and on the executor never holding user approvals. Amounts reported by a venue only size later steps.
- **User or dapp ↔ client API.** The contracts apply no policy to the route. Whether the call parameters match the route is a duty of the client and the user, with `amountOutMinimum` as the only on-chain backstop.
- **Router ↔ executor.** The router holds approvals and the reentrancy lock, and the executor holds the funds. Only the router can call `execute(...)`, but the executor has open entry points such as `receive()` and the token hooks.
- **Executor ↔ pool callbacks.** A transient commitment binds each callback to the pool of the current step. It does not cap the amount that pool may request. The plugin uses a stricter commitment that does.
- **Executor ↔ generic call targets.** The call runs with the executor as sender, so its balances and allowances are the authority exercised.
- **Executor ↔ token issuers.** Tokens with blacklists or freezes, such as stablecoins and compliance-gated tokens like WTGXX, can block the executor address if it is involved in malicious activity or receives transfers from an address that is later blacklisted. Because the address is shared, one block denies that token to every user.
- **Contracts ↔ tokens and chain.** ERC-20 tokens are expected, and the documentation presents fee-on-transfer tokens as supported through the measure flag; other behaviours are handled by convention. The contracts need transient storage and a wrapped native token, and a deployment mistake is permanent.

Threat Actors

Swapper (user or dapp)

Honest capabilities: call `swap(...)` or `swapWithPermit(...)` with a route obtained from the API.

If mistaken or deceived: an unvalidated route or a loose `amountOutMinimum` can lose that swapper's own input. The router pulls exactly `amountIn` per call, so the exposure is limited to that call and the standing approval.

Bounds: a positive minimum output, a deadline check, a reentrancy lock, and the swapper's own review of the parameters. *(Tampering / Spoofing.)*

Client API (route builder)

Honest capabilities: choose venues, splits, funding modes and call data, and pre-quote off-chain.

If compromised or wrong: it can build a route that sends value through attacker-controlled targets or to a chosen recipient. The router never learns who built the route.

Bounds: none on-chain beyond the user's parameters and the final output measurement. *(Spoofing / Tampering.)*

Adversarial venue (pool, pair, `PoolManager`, hook or generic call target)

Honest capabilities: execute a swap or funded call, call back to be paid, and return an amount.

If malicious or buggy: it can under-pay, report an unrelated amount, request a different amount in a callback, re-enter through a callback, or keep an approval granted to it. Because the final check measures a whole balance, a venue that sends tokens to the executor can also change what the check sees.

Bounds: callbacks accept only the committed caller, the plugin pays at most `amountIn`, the executor holds no standing user approvals, and the final output check runs after every step. *(Tampering / Elevation of privilege / Denial of service.)*

Opportunistic third party (observer, searcher, bystander)

Honest capabilities: call any open function, send tokens to the executor or plugin, and submit its own swap.

If malicious or opportunistic: it can collect balances left in the executor or plugin by routing its own swap, change what a later swap measures, and front-run, sandwich or reuse a permit seen in the mempool.

Bounds: a swallowed permit failure keeps a front-run permit from blocking the swap, callbacks revert unless armed, and sandwiching is bounded only by the victim's minimum output. *(Tampering / Denial of service / Information disclosure.)*

Recipient and token contracts

If hostile or unusual: a contract recipient can re-enter or revert the final payout, which blocks a swap with an ETH output. A token can credit less than requested, rebase, or require a credential held by the shared executor.

Bounds: the router lock, payout only after the output check, the measure flag for fee-on-transfer tokens, and `TransferHelper` for non-standard return values. The measure flag does not cover a fee charged on the payout itself, because the minimum is checked before that transfer. *(Denial of service / Tampering.)*

Deployer and chain operator

A wrong wrapped native token address is immutable, and a chain without transient storage breaks the lock and the callback commitments. The contracts are ownerless, non-upgradeable and have no rescue function, so there is no privileged-key threat. The deployment configuration is treated as a trust assumption.

Invariant / Violation Matrix

Invariant	How it could be violated	Mitigation / bound
The recipient receives at least `amountOutMinimum`	A route delivers less than promised, or the output token charges a fee on the payout transfer	Final balance test before payout, which bounds the total received rather than the swap's own contribution. The check runs before the payout transfer, so a fee-on-transfer output token lets the recipient receive less than the minimum by the amount of its fee
Only the swapper's own input is at risk	A route step reaches approvals or other swaps' funds	Approvals live on the router only. The shared executor still holds other flows' balances if swaps interleave
Execution cannot be re-entered	A venue, hook or recipient calls back into `swap(...)`	Transient reentrancy lock
A callback pays only the committed pool	Another caller or pool invokes a callback	Transient commitment checked against the caller. The executor does not cap the amount, the plugin does
Input is spent as the route states	Step percentages exceed 100 percent	Underflow reverts, and the last step takes the remainder
Reported step outputs do not decide payout	A venue lies to inflate later steps	Payout uses a measured balance. A lie can still distort the route's own splits
The route is well-formed before it is trusted	Truncated or inflated lengths shift later reads	Length and offset checks on executed steps. Parsing uses unchecked reads
Input matches the route's first token and amount	Parameters name a different token or amount	Not enforced on-chain, left to the client and the user
The executor holds no standing approvals	A generic call step leaves a spender allowance	Approvals are scoped per step but one funding mode does not reset them afterwards
Fee-on-transfer tokens are accounted correctly	The nominal amount overstates what arrived, or a pool receives less than the amount it priced and rejects the step	The measure flag sizes the basis from the live balance. It does not make a pool receive the full amount, so pool steps revert for these tokens
The compliance credential stays usable	The KYC NFT is moved or lost	Only a route-authored generic call can move it
The executor can receive and send a routed token	An issuer blacklists or freezes the shared executor address	Not preventable on-chain, since the executor is a single immutable address. Impact is limited to routes through that token

Table 4: Scoped invariants and principal violation paths

Trust Assumptions

The client's API builds routes honestly. Users and dapps check the non-route parameters before submitting and set `amountOutMinimum` tightly enough to limit sandwiches and hostile routes. Input tokens behave as ERC-20. The documentation presents fee-on-transfer tokens as handled through the measure flag, which covers the balance the executor holds but not the amount a pool receives. Swappers receiving a fee-on-transfer output token allow for its fee, which the minimum does not cover. The executor and plugin are not expected to hold balances between swaps, and token issuers are assumed not to blacklist the executor address. The target chains support transient storage, and the imported libraries behave as documented.

Risks Checked and Refuted

The following paths were probed and are blocked by an on-chain control. Calling `execute(...)` directly is rejected by `onlyRouter`. A foreign address cannot satisfy the callback commitment. A reentrant `swap(...)` is rejected by the transient lock. A route cannot spend a user's standing approval, because only the router pulls user funds. Step percentages above 100 percent revert. A zero `amountOutMinimum` is rejected. A front-run permit cannot block `swapWithPermit(...)`.

9 Evaluation of Provided Documentation

The audited scope consists of `FraxswapRouterMultihopV2.sol` and `PancakeV3Plugin.sol`. The client provided a repository `README.md`, extensive in-code natspec (including the route format specification in the router header), and natspec headers in the deterministic deployment scripts. During the engagement the client also described the intended trust model: routes are built by the client's API, users check the non-route parameters, `amountOutMinimum` protects against a bad route, the WTGXX whitelist works by sending a KYC NFT to the executor, and fee-on-transfer tokens are expected to work. Only part of this is mirrored in the natspec.

Strengths

- **Router and executor natspec.** The contract header explains the router and executor split, the untrusted-step model, the leftovers policy and the packed route format down to byte level, with the reasoning behind non-obvious choices.
- **Route encoder.** `FraxswapRouteEncoder` documents every helper and agrees with the layout read by the executor.
- **Plugin natspec.** The `PancakeV3Plugin` header is a self-contained integration guide: it explains why a PancakeSwap V3 pool needs the plugin, how to encode the step, how to quote it, and what the callback is bound to. The offsets given (68 for the patched amount, 0 for the returned amount) match the ABI layout of `swap(...)`.
- **Deployment scripts.** The CREATE2 scripts document the deployment proxy, how to predict the address and a short usage recipe. The router script also documents the per-chain wrapped-native constructor argument.

Missing Documentation

- **Project README.** The `README.md` is the generic `Frax Template`. It does not describe the router, the executor, the plugin or the trust assumptions, and its deployment steps point to placeholder scripts instead of the CREATE2 scripts actually used.
- **Architecture and trust model.** There is no standalone document on the call flow or the trust model; both live only in code comments. No documentation of the off-chain route builder or of a user-side checklist exists in the repository.
- **Token classes and operations.** A list of supported token classes (fee-on-transfer, rebasing, ERC-777), the WTGXX whitelist procedure and a table of revert strings are not provided.

Overall Assessment

The documentation is rated **Medium**. The natspec of the two in-scope contracts is thorough and mostly accurate, and the byte-level route format and plugin recipe are clear strengths. The main weaknesses are that the README describes none of the system, that the trust model is documented only in code comments, and that a few natspec statements disagree with or omit behaviour that the code has (see Subsection 9.1). Because, according to the client, safety relies on the client's route builder and on user-side checks, the missing documentation of both is the most valuable gap to close.

9.1 Documentation QA

The main discrepancies between the documentation and the code at commit `cca7278` are listed below. Each was checked against the code by reading.

- **Native ETH input and recipient.** The natspec says `address(0)` as `tokenIn` means native ETH, but the router also accepts native ETH when `tokenIn` is the WETH9 address and `msg.value` is set. On that path the WETH is sent to the target chosen by the direct funding bit, so the header sentence that this bit is ignored for native ETH input does not describe it. The `recipient` field also defaults to the caller when zero, which is not stated.
- **Route rules that exist only as prose.** The header says the last step must not use a receiver override and that a direct-funded first step must be encoded as already funded. Neither rule is enforced by a check or a dedicated error, and the UniV3 and UniV4 steps have no already-funded mode. Malformed routes (no swaps, an invalid `creditTo` index) panic, and this behaviour is not documented.
- **Measure flag basis.** The header describes the basis as the executor's measured `tokenIn` balance. The code reads the executor's entire balance of that token when the swap starts, which includes amounts credited to other swaps and any leftover. The documentation does not state this.
- **Fee-on-transfer support.** The documentation presents support through the measure flag. By code reading, a fee-on-transfer first input is credited at its nominal amount unless the first swap sets the flag; this was not confirmed by execution. The documentation does not say in which swaps the flag is required.

- **Approval and output-minimum claims.** The executor header says there are no approvals to steal, but generic call funding mode 1 approves a route-chosen target, and an unconsumed allowance can remain outstanding. The output minimum is presented as the swapper's protection, but it is checked against the executor's whole balance of the output token, so any balance left there counts toward it. The header says leftovers are finders keepers, but neither effect is spelled out.
- **Plugin statements.** The header says the plugin has no state, but it uses two transient variables as a lock and a mode flag. The `parseRevertReason(...)` natspec states that every real revert is empty, a 4-byte error or a selector plus encoded data, which describes Solidity-style reverts; any revert of exactly 32 bytes is read as a quote. The parameters `tokenIn` and `zeroForOne` are documented as requirements ("must be token0 or token1") but are not validated by the plugin.

Recommendations

a. Replace the template `README.md` with a project README covering the architecture, supported chains and the deployment procedure with the per-chain `WETH9`.

b. Publish the route format and trust model as a standalone document, with a user-side checklist (including how to choose `amountOutMinimum`) and validation rules for route builders.

c. Correct the natspec items above, in particular the native ETH paths, the measure flag basis, outstanding approvals and the plugin's transient state.

d. State which token classes are supported and where, and the assumption that the executor holds no output-token balance when a route starts.

10 Test Suite Evaluation

10.1 Compilation Output

```
> forge build --force
Compiling 110 files with Solc 0.8.34
Solc 0.8.34 finished in 107.75s
Compiler run successful with warnings:
warning[unsafe-typecast]: typecasts that can truncate values should be checked
...
(42 lint warnings: 37 unsafe-typecast, 3 erc20-unchecked-transfer, 2 block-timestamp)
```

The project compiles successfully. All warnings are Foundry lint notes spread over in-scope, out-of-scope and test files (nine in the V2 router and two in the Pancake plugin), and none of them changes behaviour.

10.2 Tests Output

```
> forge test -j 1 --fork-url <MAINNET_URL> --fork-retries 20 --fork-retry-backoff 2000
Ran 21 tests for src/test/FraxswapRouterAmountOutTest.sol:FraxswapRouterAmountOutTest
Suite result: ok. 21 passed; 0 failed; 0 skipped
Ran 25 tests for src/test/FraxswapRouterAmountOutV2Test.t.sol:FraxswapRouterAmountOutV2Test
Suite result: ok. 25 passed; 0 failed; 0 skipped
Ran 42 tests for src/test/FraxswapRouterMultihopTest.t.sol:FraxswapRouterMultihopTest
Suite result: ok. 42 passed; 0 failed; 0 skipped
Ran 30 tests for src/test/FraxswapRouterMultihopV2Test.t.sol:FraxswapRouterMultihopV2Test
Suite result: ok. 30 passed; 0 failed; 0 skipped
Ran 16 tests for src/test/PancakeV3PluginTest.t.sol:PancakeV3PluginTest
Suite result: ok. 16 passed; 0 failed; 0 skipped
Ran 5 test suites in 39.79s: 134 tests passed, 0 failed, 0 skipped (134 total tests)
```

All 134 tests pass in an independent run, including the 46 that target the in-scope contracts.

10.3 Coverage

```
> forge coverage --report summary --ir-minimum -j 1 --fork-url <MAINNET_URL> \
    --match-contract "FraxswapRouterMultihopV2Test|PancakeV3PluginTest"
Ran 2 test suites: 46 tests passed, 0 failed, 0 skipped (46 total tests)
| File                                    | % Lines          | % Statements     | % Branches     | % Funcs        |
| src/contracts/FraxswapRouterMultihopV2.sol | 84.71% (205/242) | 84.42% (233/276) | 36.73% (36/98) | 88.24% (30/34) |
| src/contracts/PancakeV3Plugin.sol          | 88.24% (30/34)   | 87.50% (28/32)   | 40.00% (6/15)  | 100.00% (4/4)  |
```

Coverage was measured once, for the two in-scope test contracts and the two in-scope source files only. Line and function coverage are good, but branch coverage is low for both files (36.73% and 40.00%), which means most guard and flag branches are not exercised. The figures are indicative, because `-ir-minimum` compiles without the optimiser and distorts source mappings.

10.4 Overall Strengths

- **Real-venue integration.** Routes are executed against live Fraxswap, Uniswap V2/V3/V4, Curve, Pendle, Sky Lite PSM and KyberSwap contracts, which validates the packed route encoding against real calldata rather than against mocks.
- **Outcome-oriented assertions.** Split and merge routes, partial fills and the executor holding no residue are asserted on outcomes, not only on the absence of reverts.
- **Adversarial coverage of the riskiest surfaces.** Dedicated tests cover approval theft through the generic call step, a malicious V3 pool, unauthorised callers and fee-on-transfer inputs.

10.5 Cross-Cutting Weaknesses

- **No fuzz, invariant or property testing.** There is no `testFuzz`, `invariant_`, `bound(...)` or `StdInvariant` anywhere, although the executor parses a hand-packed route format that the caller controls.
- **Happy paths dominate.** Most revert and flag branches are not driven, and two revert tests use a bare `vm.expectRevert()`, so they would also pass for an unrelated failure.

- **Payer and recipient are always the same address.** In every V2 test the payer and the recipient are the same account, so a recipient that differs from the payer is never exercised.
- **The test setup is undocumented.** The tests need archive-state RPCs for three chains, but `example.env` lists only `FRAXTAL_MAINNET_URL` and the README does not document `MAINNET_URL` or `POLYGON_MAINNET_URL`.

10.6 Specific Untested Paths

The following live paths and branches are never driven by the suite:

- **Router**: deadline expiry (`"Transaction too old"`); the router reentrancy guard (the plugin's own guard is tested); the ETH accounting reverts for `msg.value` mismatches and unused ETH (`FSR:II`, `FSR:IE`); and the default recipient when it is the zero address.
- **Executor, Uniswap V4 and V3 steps**: the V4 callback authentication and token check (`FSE:V4Auth`, `FSE:V4TokenIn`) and the V3 zero-delta check (V3 callback authentication is tested).
- **Executor, generic call step**: the bounds and value rules (`FSE:PatchOOB`, `FSE:ReturnOOB`, `FSE:ValueToken`) and the unknown step type revert (`FSE:StepType`).
- **Executor, UniV2 step**: the receiver override flag (the custom fee flag is tested).

10.7 Conclusion

The delivered suite gives meaningful real-world evidence for the main execution paths and for the PancakeSwap V3 plugin, and the adversarial tests target the right surfaces. It is held back by three gaps:

a. Fuzz and invariant testing is absent, although the route format is attacker-controlled.

b. Branch coverage is low and many guard reverts are never driven.

c. The recipient is never distinct from the payer, and the test setup is not documented.

The test suite is assessed as **Medium**. To close the gaps, **CODESPECT** recommends:

- Fuzz tests over the route parser, and an invariant that the executor holds no tokens after a successful swap.
- Exact-revert tests for the untested guards listed above, replacing the bare `vm.expectRevert()` calls with specific errors.
- Tests with a recipient different from the payer, and a non-fork unit layer with mock pools.
- Documentation of the three required RPC variables and of a working test invocation.